Workflow for tidyverse-side development without losing parity

2026-04-15

Author. pmsimstats team

Purpose. This note captures the recommended workflow for editing `implementations/tidyverse/` while preserving numerical parity with `implementations/original-extended/`. The cross-implementation parity test at `analysis/scripts/parity/test-parity-extended-tidyverse.R` is the safety net; this document describes how to integrate it into day-to-day development rather than treat it as an after-the-fact audit. Statements about the parity script were checked against its source on 2026-10-07 (inspected); the run times quoted below were not re-measured.

Contents

Notation index and glossary 3

1. Baseline before you start 3

2. Pick a tight inner loop 3

3. Edit semantics versus edit shape 4

4. When parity breaks mid-refactor 4

5. Scope edits to one concern at a time 5

6. Guardrails around parallelization and RNG 5

7. Expand the parity test as coverage gaps appear 5

8. When divergence is intentional 5

9. Before merging 6

Checklist 6

Optional infrastructure to land 7

Notation index and glossary

This note uses no mathematical symbols. Code identifiers appear in backticks only (`c_bm`, `t_half`, `n_cores`, `Dbc`).

- **Parity.** Numerical agreement between the `original-extended` (data.table) and `tidyverse` implementations under matching seeds, to the script's tolerances: 10^{-10} on the covariance matrix and the generated data, 10^{-6} on the interaction estimate and its standard error, 10^{-8} on its p-value.
- **Cell.** One combination of design, data-generating architecture (`mvn` or `mean_moderation`), carryover half-life, `c_bm` and seed in the parity grid.
- **Layer.** One of the compared stages: covariance matrix (Sigma), generated data, analysis, and a batch smoke test.
- **Baseline.** A saved parity report (`parity_baseline.rds`) against which later reports are compared.
- **Expected divergence.** A cell-layer failure annotated as intentional in `analysis/scripts/parity/parity_divergences.csv`.

1. Baseline before you start

Run the full parity sweep once on the clean branch and save the report as the frozen "this is what parity looks like before I touched anything" artifact:

```
git checkout -b tidyverse-refactor
Rscript analysis/scripts/parity/test-parity-extended-tidyverse.R
cp analysis/scripts/parity/parity_report.rds analysis/scripts/parity/parity_baseline.rds
```

Commit `parity_baseline.rds` with the branch. At the end of the development cycle, diff the current `parity_report.rds` against it to confirm the only differences (if any) are the ones you intended.

2. Pick a tight inner loop

The default mode runs in approximately 10 seconds on current hardware. That is fast enough to run after every substantive edit. Use `--quick` (approximately 2 seconds, 16 cells) while iterating on a specific function and escalate to the full sweep before committing. The default mode and `--full` run the same grid, 144 cells (four designs, two architectures, three half-lives, three values of `c_bm`, two seeds); `--full` differs only in the mode label stored in the report (inspected, 2026-10-07). The timings are from 2026-04-15 and were not re-measured.

A git pre-commit hook can enforce this, or the habit can be built manually:

```
# pre-commit hook (optional)
Rscript analysis/scripts/parity/test-parity-extended-tidyverse.R --quick || \
  { echo 'parity broken; run the full sweep for details'; exit 1; }
```

No such hook is installed in the repository as of 2026-10-07 (`.git/hooks/` holds only the sample hooks; inspected).

3. Edit semantics versus edit shape

Parity is preserved if a change is "same output, different code." It is broken if the change is "different output, possibly better." Be deliberate about which is intended:

- **Pure refactors** (rename, split function, swap `dplyr` for `purrr`, extract helper): parity *must* hold. If it breaks, a bug has been introduced.
- **Feature additions** that extend behavior without changing the existing path (new optional argument with default value, new analysis option): parity *must still hold* for the default-argument call paths the test exercises.
- **Behavior changes** (different DGP parameterization, different MVN-draw ordering for speed, different Sigma caching scheme): parity *will* break by design. Before making such a change, decide whether this also means breaking parity with `original-extended` (in which case the parity test becomes the wrong oracle) or whether the change should be applied to both implementations in lockstep.

In practice, most development falls into the first two categories. Treat the third as a separate project with explicit scope.

4. When parity breaks mid-refactor

The test reports which layer broke. Diagnose in this order:

1. **Sigma failure:** the change likely touched `build_sigma_matrix` or `build_correlation_matrix`. The BM-BR correlation branch is the most common culprit, as the earlier `dgp_architecture` scoping bug demonstrated.
2. **Data failure with Sigma passing:** the change touched the MVN sampling path, the post-draw mean-moderation shift, or the participant/path assembly logic.
3. **Analysis failure with data passing:** the change touched formula construction, `Dbc` computation, or the `nlme::lme` invocation in `lme_analysis`.

The test prints a pass count per layer and, on failure, the table of unexpected failing cells for each layer, then exits with status 1. The first row of that table is usually enough to reproduce the divergence interactively. A one-liner that compares a single numeric column across implementations is faster than re-running the full sweep during debugging.

5. Scope edits to one concern at a time

The `--quick` run covers 16 cells in approximately 2 seconds. Re-running after every approximately 20 lines of changes localizes bugs to the edit that introduced them. The alternative, batching 200 lines of changes before debugging a dozen parity failures at once, is substantially slower.

6. Guardrails around parallelization and RNG

The parity test calls `set.seed()` at the per-cell level. If a refactor consumes random draws in a different order (for example, changing `purrr::map` to `furrr::future_map` or reordering per-path MVN calls), parity will break even though the code is semantically correct. Two defences apply:

- Do not change RNG-consuming loops unless `original-extended` is updated in the same commit.
- Keep `n_cores = 1` for any parity-relevant comparison. Introduce parallelism only after functional parity is locked in.

7. Expand the parity test as coverage gaps appear

When a feature is added that the current grid does not exercise (for example, `carryover_form = 'weibull'` or an analysis option toggling random slopes), add a row to the `edge_cells` tibble in the script's edge-cell section, just before the summary. It currently holds five cells (zero `c_bm` with carryover under each architecture, two small-N cells and one empirical draw; inspected). The test grows with the surface area it protects. A one-line edit per new feature is inexpensive, and the habit compounds.

8. When divergence is intentional

If the intent is to change tidyverse behavior in a way that `original-extended` should not follow (for example, fixing a bug that lives only in the data.table implementation, or using the tidyverse implementation as a testbed for a new feature), document the divergence explicitly:

- Record *which cells* of the parity test are allowed to fail and *why*: add a row (design, architecture, half-life, `c_bm`, seed, layer, reason) to `analysis/scripts/parity/parity_divergences.csv`, and say so in the commit message.
- The script reads that file and counts matching failures as expected divergences, so the summary reports them separately and they do not set a failing exit status. (Update, 2026-10-07: this mechanism was proposed here as an `expected_divergent` flag and a `DIVERGENCES.md` file; it has since been implemented as the CSV annotation file; inspected.)
- Re-baseline after the intentional divergence lands: `cp analysis/scripts/parity/parity_report.rds analysis/scripts/parity/parity_baseline.rds`.

This keeps "expected" and "broken" visibly separated. The trap to avoid is letting an intentional divergence mask a subsequent unintentional one.

9. Before merging

Run the full sweep (not `--quick`) one final time. If the diff between `parity_report.rds` and `parity_baseline.rds` is empty (or matches documented expected divergences), the branch is safe to merge. Otherwise, the diff identifies exactly which cells regressed, which is a tighter feedback loop than "tests pass locally but something feels off."

The diff script does this and exits 0 when the reports agree:

```
Rscript analysis/scripts/parity/parity-diff.R
```

A one-liner for the same comparison of the main grid:

```
a <- readRDS('analysis/scripts/parity/parity_baseline.rds')
b <- readRDS('analysis/scripts/parity/parity_report.rds')
all.equal(a$grid, b$grid)  # TRUE, or a character vector of diffs
```

Compare reports produced in the same mode: a `--quick` report has 16 grid rows and a default or `--full` report 144, so a quick report never matches a full baseline.

Checklist

- ☐ Copy `parity_report.rds` to `parity_baseline.rds` (both in `analysis/scripts/parity/`) at the start of the branch.
- ☐ `--quick` after every focused edit (~2 s).
- ☐ Full sweep before every commit (~10 s).

- ☐ If parity breaks, read the failing-cell table; fix before proceeding.
- ☐ Never change RNG consumption order in a pure refactor.
- ☐ Add edge cells to the script when new features are added.
- ☐ Record any intentional divergence in `parity_divergences.csv` and the commit message.

Optional infrastructure to land

If the recommendations above prove useful in practice, three pieces of supporting infrastructure can be promoted from recommendation to code in the repository. Status as of 2026-10-07 (inspected):

1. A pre-commit hook that runs `--quick` and blocks the commit on failure. Not installed.
2. An annotation mechanism in the parity script so intentional divergences do not read as regressions. Implemented as `parity_divergences.csv` (Section 8).
3. A baseline-diff wrapper script (`Rscript analysis/scripts/parity/parity-diff.R`) that reports which cells differ between the current report and the baseline. Implemented (Section 9).